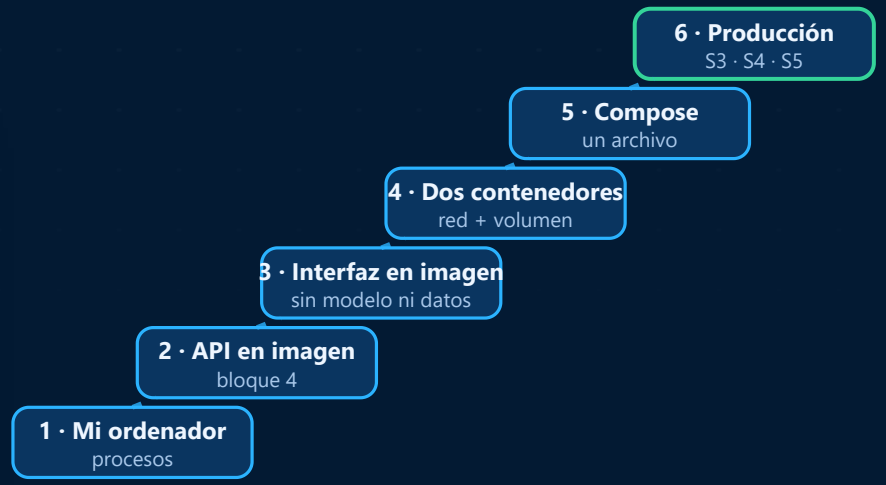
S2 · ML E IA: DE DESARROLLO A PRODUCCIÓN

Del ordenador al servicio

La API y la interfaz, paso a paso: de dos procesos en tu portátil a dos contenedores que se hablan, y lo que falta para producción.

6 ESCALONES · UNOS 60 MIN · CODESPACE · PROBADO CON DOCKER REAL



Cómo seguirla

Cada orden lleva ⧉ para copiarla en la terminal de tu Codespace, de una en una. ← → avanzan · **O** índice · **F** pantalla completa. Las salidas que ves son **reales** (GitHub Actions y Windows) y las capturas, de un **Codespace real del curso** (14/11/2026). Todo está explicado en texto en `04-Proyecto-Base/docs/del-ordenador-al-servicio.md`.

LO QUE NOS PREGUNTARON

Ya tengo la API… ¿y ahora quién la usa?

La API pide esto

```
{"recencia_dias": 35, "frecuencia": 6, "monto_total": 1250.4,
 "ticket_medio": 208.4, "unidades_total": 410,
 "productos_distintos": 38, "antiguedad_dias": 310,
 "frecuencia_90d": 2, "monto_90d": 380.5,
 "frecuencia_180d": 4, "dias_medios_entre_compras": 55.0,
 "devoluciones": 1, "es_reino_unido": 1}
```

Trece variables. Nadie de negocio las conoce.

Quien hace la campaña tiene esto

`13599, 18108, 13306`

…y quiere saber **qué hacer** con cada uno.

Hoy, en cuatro partes

- **A** · ¿por qué llevar un modelo a producción?
- **B** · todo en mi ordenador
- **C** · dos contenedores que se hablan
- **D** · ¿esto ya es producción?

El modelo que se queda en el portátil

Lo bueno (y es legítimo)

- **Rápido y barato** para explorar: sin infraestructura ni permisos.
- **Libertad** para cambiarlo todo mientras aún no sabes qué funciona.
- Suficiente para un **análisis puntual**: una decisión, un informe.
- Sirve para **validar la idea** antes de invertir en llevarla a producción.

Lo que cuesta no llevarlo

- **Nadie más lo usa**: el valor se queda en una presentación.
- «**En mi máquina funciona**»: otra persona no obtiene lo mismo.
- Depende de **una persona**: si se va o se enferma, se va el modelo.
- Se ejecuta **a mano**: pasos olvidados, versiones distintas.
- **Nadie vigila** si se degrada con datos nuevos.
- **Datos de clientes en portátiles** y sin rastro de qué modelo decidió qué.

EL OTRO LADO

Producción tampoco es gratis: ¿cuándo merece la pena?

Lo que añade producción

- Infraestructura que se **paga** cada mes.
- **Mantenimiento**: dependencias, parches de seguridad, versiones.
- **Vigilancia**: registros, métricas, alertas y alguien que responda.
- **Disciplina**: pruebas, revisiones y despliegues controlados; cambiar es más lento.

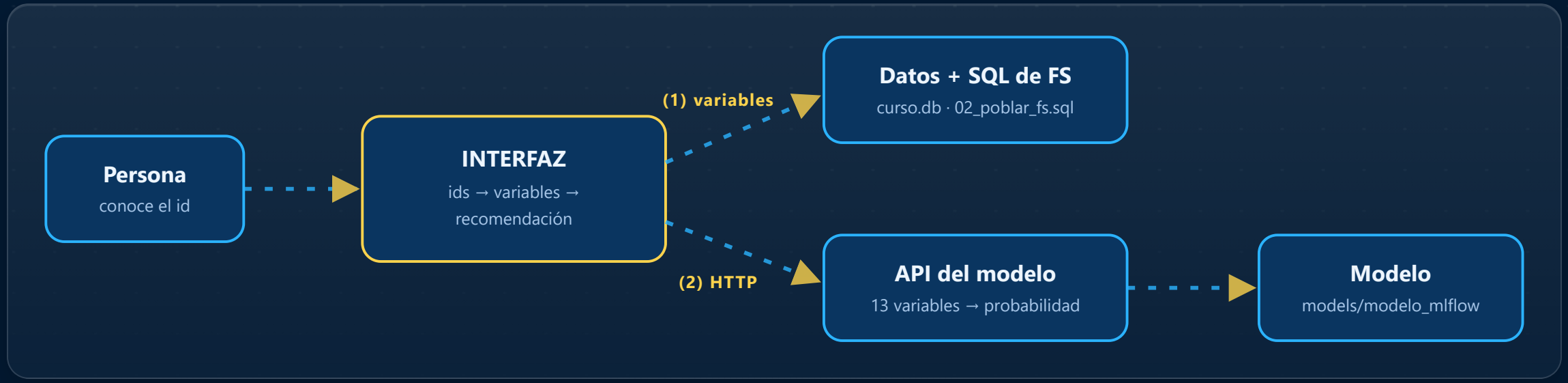
Merece la pena si respondes «sí» a alguna

PREGUNTA	SI ES «SÍ»…
¿Lo usarán **otras personas o sistemas**?	necesita una forma de llamarlo
¿Se decide con él **a menudo** (cada día, cada semana)?	no puede depender de alguien a mano
¿Los **datos cambian** y hay que volver a puntuar?	necesita un proceso repetible
¿Hay que poder **explicar qué modelo decidió qué**?	necesita versiones y trazas
¿Un fallo **cuesta dinero** o clientes?	necesita vigilancia

«Producción» no siempre es una API: a veces es un **proceso por lotes** que deja la lista de la campaña cada lunes (el laboratorio extra, `python -m ia_mlops.explotacion`).

EL MAPA

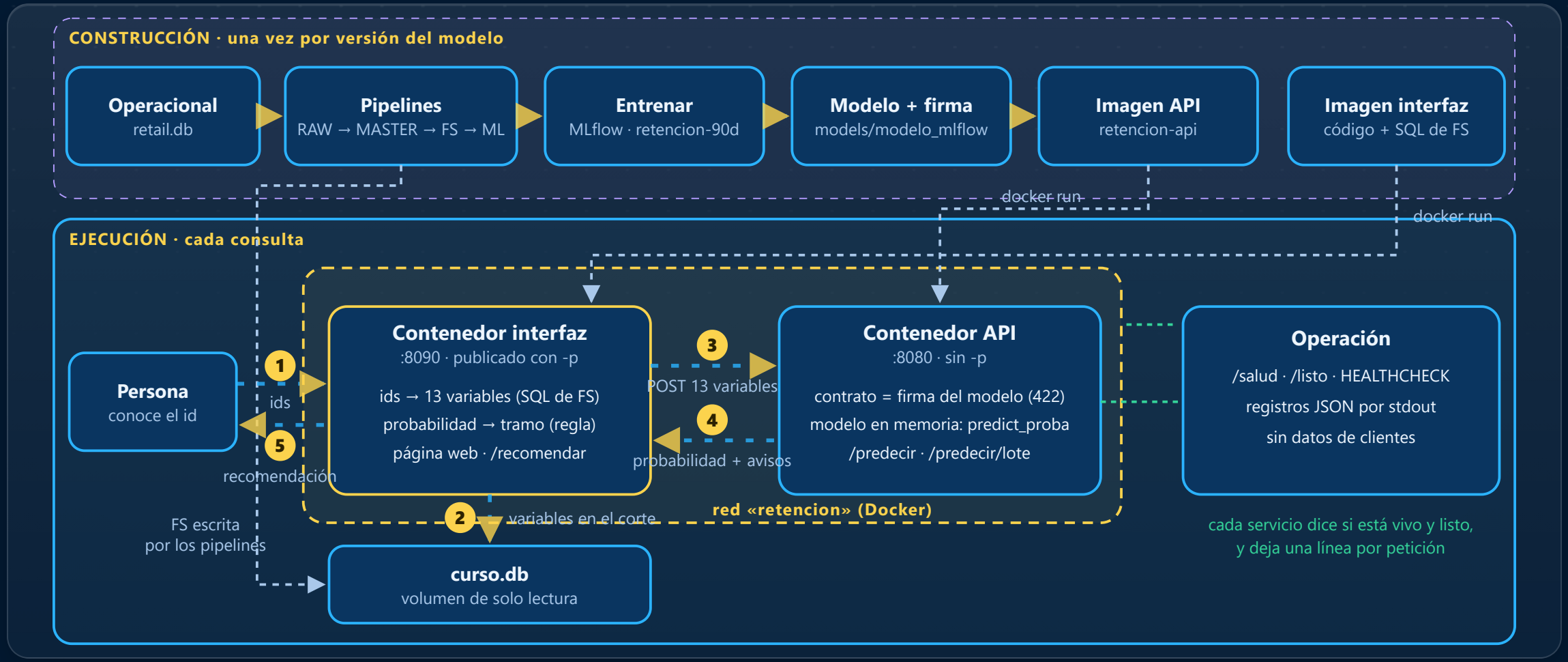
Cuatro piezas y quién usa cada una



La **API no sabe qué es un id**: solo sabe de variables. La **persona no sabe qué son las variables**: solo conoce el id.

La **interfaz traduce**: (1) calcula las 13 variables con el SQL de FS, (2) llama a la API, (3) aplica la regla de negocio de tramos.

Cómo funciona la exposición del modelo



CONCEPTOS · 1

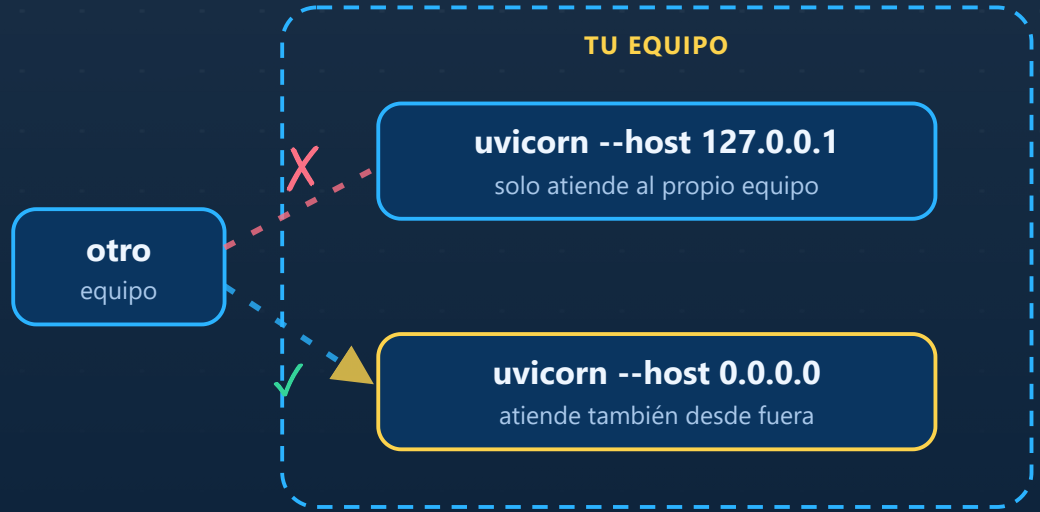
Proceso, puerto y quién puede llamar

Proceso y puerto

Un **proceso** es un programa en marcha. Un servidor se queda **escuchando** en un **puerto**, una «puerta» numerada del equipo: la API en el `8080`, la interfaz en el `8090`. Dos procesos no comparten puerto: `address already in use`.

Cliente y servidor

El **cliente** pide (navegador, `curl`), el **servidor** responde. Hablan **HTTP** con **JSON** y un código: **200** bien, **422** petición sin sentido, **503** ahora no puedo. La interfaz es servidor para ti y **cliente** de la API.



CONCEPTOS · 2

Lo que el código no lleva dentro

Dependencias

Todo lo que el programa necesita y **no está en su código**: Python 3.11, fastapi y pandas, `curso.db`, el SQL de FS… y **otro servicio**, la API. En tu portátil «está ahí» porque lo instalaste tú. **Esa es la trampa.**

Configuración por variables de entorno

VARIABLE	MI ORDENADOR	CONTENEDOR
CURSO_URL_API	http://127.0.0.1:8080	http://api:8080
CURSO_DB	data/processed/curso.db	/datos/curso.db
CURSO_DB_SOLO_LECTURA	(vacía)	1

El mismo código; cambia solo lo que se le pasa desde fuera.

Vivo no es lo mismo que listo

`/salud` ¿El proceso **responde**? Si no, se reinicia. Es lo que pregunta el HEALTHCHECK.

`/listo` ¿Tiene lo que necesita (la base y la API)? Si no, **reiniciar no sirve**: el 503 dice qué falta.

ASÍ SE VE EN TU CODESPACE · ANTES DE EMPEZAR

Prepara el Codespace: datos, modelo y Docker

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
  referencia_sin_aprender    AUC valid 0.500 | test 0.500 | precision media test 0.404
  regresion_logistica        AUC valid 0.698 | test 0.747 | precision media test 0.659
  gradient_boosting          AUC valid 0.663 | test 0.735 | precision media test 0.648
/opt/conda/envs/env-ia-mlops/lib/python3.11/site-packages/mlflow/types/utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). In
teger columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats an
d will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training da
taset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing value
s. See `Handling Integers With Missing Values <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>`_ for more detail
s.
  warnings.warn(
2026/11/14 11:52:33 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these f
ormats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is t
he 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
Successfully registered model 'retencion-90d'.
Created version '1' of model 'retencion-90d'.
/opt/conda/envs/env-ia-mlops/lib/python3.11/site-packages/mlflow/types/utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). In
teger columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats an
d will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training da
taset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing value
s. See `Handling Integers With Missing Values <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>`_ for more detail
s.
  warnings.warn(
2026/11/14 11:52:39 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these f
ormats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is t
he 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
  Mejor modelo: regresion_logistica. Guardado en /workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base/models/modelo.joblib
  Registrado en MLflow como «retencion-90d», version 1. Para verlo: python -m ia_mlops.mlflow_ui
  (17.7 s)
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

```
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $ docker version --format 'Docker {{.Server.Version}
}' && docker image ls
Docker 29.9.3

                                                                                                    i Info →  U In Use

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   cf1b20a0c89a       25.9kB         9.49kB
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Terminal del Codespace · solo la primera vez

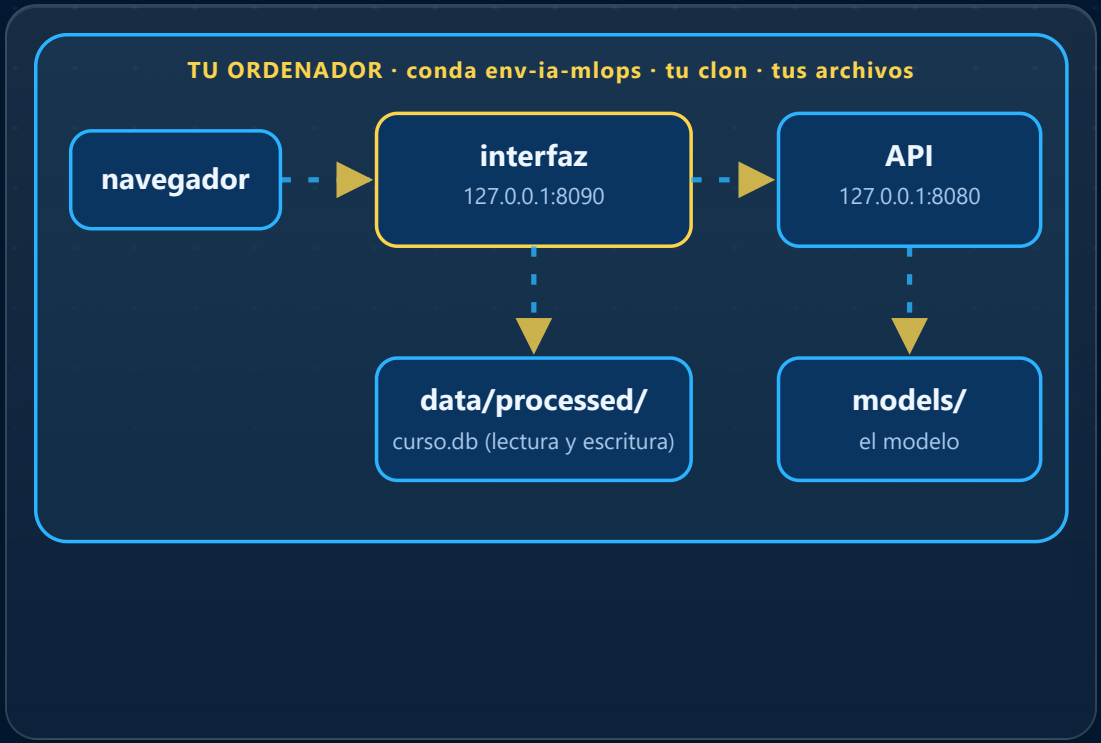
```
$ cd 04-Proyecto-Base
$ python -m ia_mlops.operacional
$ python -m ia_mlops.pipeline
$ docker version --format 'Docker {{.Server.Version}}'
```

Qué mirar

- Un Codespace **nuevo** no trae `retail.db`: sin `operacional`, el pipeline no tiene de dónde leer.
- El pipeline acaba con **«Mejor modelo: regresion_logistica»** y **«version 1»** en MLflow.
- Los avisos `UserWarning` y `WARNING mlflow` son de MLflow: no son errores.
- `docker version` responde: el Docker del Codespace está listo.

ESCALÓN 1 · PRÁCTICA

Todo en mi ordenador



Dos terminales · con conda activate env-ia-mlops · desde 04-Proyecto-Base

```
# terminal 1: la API del modelo
$ python -m uvicorn ia_mlops.api.app:app --host 127.0.0.1 --port
  8080
# terminal 2: la interfaz, que llama a esa API
$ python -m ia_mlops.interfaz --url http://127.0.0.1:8080
```

Comprobar

```
$ curl http://127.0.0.1:8090/salud
$ curl http://127.0.0.1:8090/listo
```

SALIDA REAL · WINDOWS · 14/11/2026

```
{"estado":"listo","datos":"ok","modelo":"http://127.0.0.1:8080"}
```

ASÍ SE VE EN TU CODESPACE · ESCALÓN 1

Tres terminales: API, interfaz y comprobación

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026



1 · Izquierda: la API

`Uvicorn running on http://127.0.0.1:8080` y la línea JSON `modelo_cargado`.

2 · Centro: la interfaz

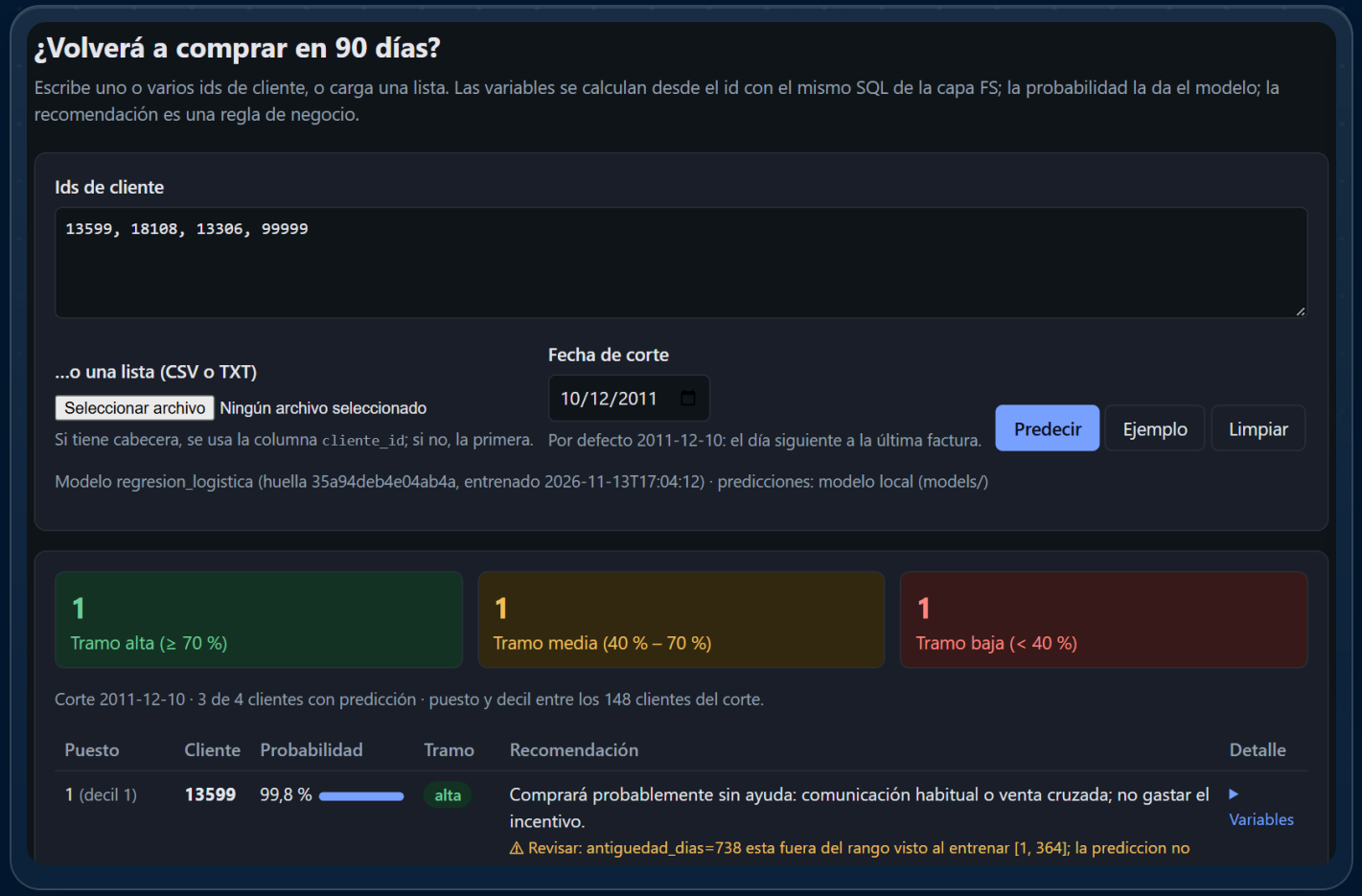
Divide el terminal con **Ctrl+Shift+5** (o el icono ◫). Sale `Interfaz en http://127.0.0.1:8090`.

3 · Derecha: comprobar

Otra división para los `curl`: `/salud` y `/listo` en 200, y en el centro se ve cada petición.

ESCALÓN 1 · PRÁCTICA

La interfaz: ids → probabilidad → recomendación



Prueba en la página

- `13599` → 99,8 %, `alta`, puesto 1 de 148
- **Ejemplo**: tres tramos y un id que no existe
- Carga un **CSV** con columna `cliente_id`
- Corte `2011-03-01`: ¿por qué cambian las variables?

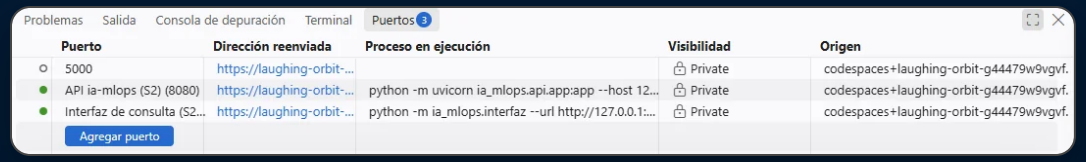
Para pensar

Casi todos salen ⚠ **Revisar**: `antiguedad_dias=738` fuera de lo visto al entrenar `[1, 364]`. El modelo se entrenó con cortes de 2010. **Así se ve un modelo que nadie vigila** (deriva, S5).

ASÍ SE VE EN TU CODESPACE · ESCALÓN 1

Abrir la página: pestaña Puertos → 8090

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026



Qué mirar

- La pestaña **Puertos** lista el **8080** (API) y el **8090** («Interfaz de consulta (S2)»), con su proceso.
- Visibilidad **Private**: solo tu cuenta de GitHub puede abrir esa dirección.
- Pulsa la **dirección reenviada** del 8090 (o el icono del globo) y se abre la página.
- Abajo: «predicciones: **http://127.0.0.1:8080**» — en el escalón 1 la interfaz llama a la API por `127.0.0.1`.



ESCALÓN 1 · CONCLUSIÓN

«En mi máquina funciona»… y solo ahí

DEPENDE DE…	¿QUÉ PASA EN OTRO EQUIPO?
Tu **entorno conda** y sus versiones	Otras versiones, otras probabilidades… o no arranca
Tus **archivos** (`models/`, `curso.db`)	Si no ejecutó el pipeline, no existen
Tu **sistema operativo**	Cambian comandos y comillas (PowerShell o bash)
Tu **sesión abierta**	Cierras el portátil y el servicio desaparece
`127.0.0.1`	Nadie más puede llamarlo

Prueba vivo frente a listo

Para la API (Ctrl+C en la terminal 1): `/salud` sigue en 200 y `/listo`:

SALIDA REAL · WINDOWS · 14/11/2026

```
{"detalle":"La API del modelo en
http://127.0.0.1:8080 no responde (…). ¿Está
arrancada y /listo responde?"}
```

Sirve para **desarrollar** y **demostrar**. No es un servicio: nadie puede depender de él.

CONCEPTOS · 3 · REPASO

Lo mínimo de contenedores para seguir

Imagen
Plantilla congelada: base, Python, paquetes y código. No cambia.

Contenedor
Una imagen **en ejecución**. Al borrarlo se pierde lo que escribió dentro.

Contexto y `.dockerignore`
Lo que se envía a Docker al construir, y lo que **no**.

`-p equipo:contenedor`
Abre una puerta del equipo hacia el contenedor. Sin `-p`, no se llega.

`-v origen:destino:ro`
Monta una carpeta del equipo dentro, en **solo lectura**. Los datos quedan fuera de la imagen.

Red
Los contenedores de una red se encuentran **por su nombre**: Docker hace de guía telefónica.

`-e` / `ENV`
La configuración, desde fuera.

`USER 10001`
Sin privilegios: si alguien lo ataca, puede hacer menos.

`HEALTHCHECK`
Docker pregunta «¿estás vivo?» y `docker ps` muestra `(healthy)`.

ESCALÓN 2 · REPASO DEL BLOQUE 4

La API ya vive en una imagen

Lo que hiciste en el bloque 4

```
$ docker build -f deploy/docker/Dockerfile -t retencion-
  api:local .
```

Lleva

Python 3.11, versiones exactas (`requirements-api.lock`), `ia_mlops/` y **el modelo**.

No lleva

Datos de clientes, cuadernos, pruebas ni SQL (`.dockerignore` de la raíz).

Qué ganamos

Ya no depende de tu conda ni de tu sistema operativo: cualquier equipo con Docker obtiene **lo mismo**, hasta el último decimal (lo comprobaste con `comparar`).

Qué falta

Sigue pidiendo **13 variables**. Falta la pieza para personas: la interfaz, en su propia imagen.

ESCALÓN 3 · DISEÑO

¿Por qué una imagen aparte para la interfaz?

1

Sin datos de clientes en la imagen del modelo

Si la API buscara las variables por id, `curso.db` viajaría dentro y cualquiera con la imagen tendría los datos.

→

2

Cada pieza cambia por un motivo

Reentrenas el modelo sin tocar la interfaz; cambias la página o los tramos sin reconstruir el modelo.

→

3

Cada pieza escala por separado

En producción puede haber diez copias de la API y una sola interfaz.

ESCALÓN 3 · INTERFAZ.DOCKERFILE

La receta, línea a línea

```
FROM python:3.11-slim
```

```
ENV CURSO_DB=/datos/curso.db \
    CURSO_DB_SOLO_LECTURA=1 \
    CURSO_URL_API=http://api:8080
```

```
COPY requirements-api.lock requirements-interfaz.txt ./
RUN pip install -c requirements-api.lock \
    -r requirements-interfaz.txt
```

```
COPY ia_mlops/ ia_mlops/
COPY sql/fs/02_poblar_fs.sql sql/fs/…
```

```
RUN useradd --uid 10001 … && mkdir /datos
USER 10001
```

```
EXPOSE 8090
HEALTHCHECK … /salud
```

```
CMD ["uvicorn", "ia_mlops.interfaz.app:app",
     "--host", "0.0.0.0", "--port", "8090"]
```

3. Dependencias

Solo fastapi, uvicorn, pydantic, pandas y numpy. -c es un archivo de RESTRICCIONES: no instala nada por sí mismo, fija la versión de lo que se instala. Las dos imágenes usan las mismas versiones.

> Sin scikit-learn ni mlflow: la interfaz no ejecuta el modelo.

ESCALÓN 3 · PRÁCTICA

Construir y comprobar que no lleva nada de más

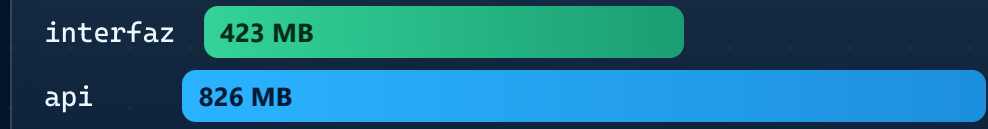
Terminal del Codespace · desde 04-Proyecto-Base

```
$ docker build -f deploy/docker/interfaz.Dockerfile -t retencion-interfaz:local .
$ docker image ls 'retencion-*'
$ docker run --rm --entrypoint sh retencion-interfaz:local -c 'find /app -type f -not -path "*/ia_mlops/*"'
$ docker run --rm --entrypoint python retencion-interfaz:local -c "import sklearn"
```

Cada imagen, su lista de exclusiones

`interfaz.Dockerfile.dockerignore`: Docker la usa en lugar de la de la raíz. Estrategia: **excluir todo** (`*`) y dejar pasar solo lo que se copia (`!ia_mlops`, `!sql/fs/02_poblar_fs.sql`…).

Tamaño



La mitad: sin scikit-learn, scipy ni mlflow.

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
/app/requirements-api.lock
/app/requirements-interfaz.txt
/app/sql/fs/02_poblar_fs.sql
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
ModuleNotFoundError: No module named 'sklearn'
```

ASÍ SE VE EN TU CODESPACE · ESCALÓN 3

Construir las dos imágenes y mirar dentro

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
sha256:438c18b7f5ca0c0420af9c7db108360f2bc9dab1c99e4f162fef52938779a438
[+] Building 28.1s (11/11) FINISHED                                    docker:default
 => [internal] load build definition from interfaz.Dockerfile                   0.1s
 => => transferring dockerfile: 2.87kB                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim             0.4s
 => [internal] load build context                                               0.0s
 => => transferring context: 6.08kB                                             0.0s
 => [1/7] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7a7b535   0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7a7b535   0.0s
 => CACHED [2/7] WORKDIR /app                                                   0.0s
 => [3/7] COPY requirements-api.lock requirements-interfaz.txt ./               0.3s
 => [4/7] RUN pip install -c requirements-api.lock -r requirements-interfaz.txt  17.9s
 => [5/7] COPY ia_mlops/ ia_mlops/                                              0.1s
 => [6/7] COPY sql/fs/02_poblar_fs.sql sql/fs/02_poblar_fs.sql                  0.0s
 => [7/7] RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser && mkdir /datos   0.3s
 => exporting to image                                                          9.0s
 => => exporting layers                                                         7.3s
 => => exporting manifest sha256:6cbb0dd6ba7e05e37693c2e99b1c059478ef41cdf0eea86292387ff47b7a1067   0.0s
 => => exporting config sha256:28b2339d2cd16328eab26b3b6102321b54e2419713c143d591f48072136caeed   0.0s
 => => exporting attestation manifest sha256:093ddd28f21fcce0cd685a5ce8a78529d13d7ba59951344e0d10a7c9c4a30918   0.0s
 => => exporting manifest list sha256:bbd9de1422ed272334fd996455ca3f58e9b4e794b496ef797fdb9433b3e902db   0.0s
 => => naming to docker.io/library/retencion-interfaz:local                     0.0s
 => => unpacking to docker.io/library/retencion-interfaz:local                  1.6s
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

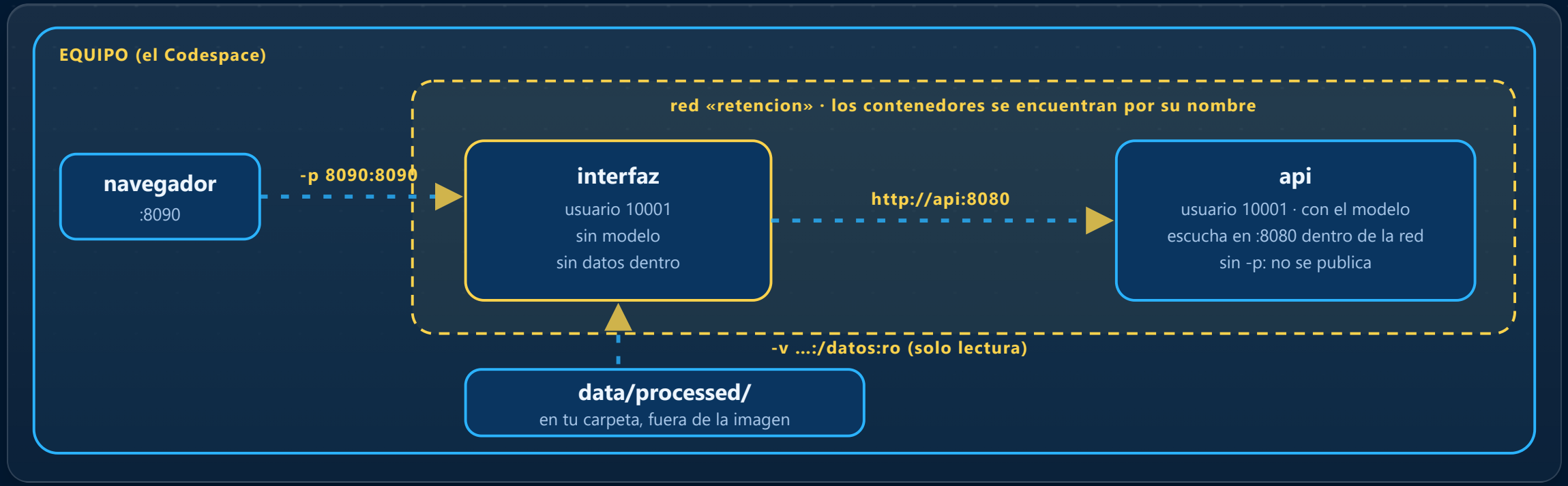
```
IMAGE                       ID             DISK USAGE   CONTENT SIZE   EXTRA
retencion-api:local         438c18b7f5ca       826MB          183MB
retencion-interfaz:local    bbd9de1422ed       423MB         96.1MB
/app/sql/fs/02_poblar_fs.sql
/app/requirements-api.lock
/app/requirements-interfaz.txt
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'sklearn'
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Qué mirar

- Antes, sin parar: **Ctrl+C** en la API y en la interfaz del escalón 1 (liberan el 8080 y el 8090).
- El build de la interfaz tiene **7 pasos**: el más largo es `pip install -c requirements-api.lock …` (unos 18 s).
- `docker image ls`: **826 MB** la API, **423 MB** la interfaz. Igual que en GitHub Actions.
- Dentro de `/app`, fuera del código, solo **tres archivos**: ni modelo ni datos.
- `import sklearn` termina en **`ModuleNotFoundError`**: es lo que queremos.

ESCALÓN 4 · EL SERVICIO

Dos contenedores que se hablan



Red: la interfaz llama a `http://api:8080`, por **nombre**.

Puertos: solo la interfaz tiene `-p`. La API no se publica.

Datos: un volumen `:ro`. Se leen; no se pueden tocar.

ESCALÓN 4 · PRÁCTICA

Paso a paso: red, API e interfaz

Terminal del Codespace · desde 04-Proyecto-Base

```
# 1. la red
$ docker network create retencion
# 2. la API: sin -p, no se publica
$ docker run -d --name api --network retencion retencion-api:local
$ docker ps
# 3. la interfaz: su puerto, sus datos en solo lectura
$ docker run -d --name interfaz --network retencion -p 8090:8090 -v
  "$PWD/data/processed:/datos:ro" retencion-interfaz:local
# 4. comprobar
$ curl http://127.0.0.1:8090/listo
```

Espera a que `docker ps` muestre `(healthy)` en la API. En PowerShell (Docker Desktop, sin probar): `-v "${PWD}\data\processed:/datos:ro"`.

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
{"estado":"listo","datos":"ok","modelo":"http://api:8080"}
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
NAMES      STATUS                   PORTS
interfaz   Up 6 seconds (healthy)   0.0.0.0:8090→8090/tcp
api        Up 6 seconds (healthy)   8080/tcp
```

Fíjate en los puertos: la interfaz tiene `0.0.0.0:8090→8090` (publicado); la API solo `8080/tcp`: escucha en su red, sin puerta al equipo.

ASÍ SE VE EN TU CODESPACE · ESCALÓN 4

Red, API y interfaz: los dos sanos

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
bdb61d10bcde973e0a3921f92e8e1b0112e69c782b059661e98e5952d8af35d2
36b598270dc7dc4577ec52b5a6b39474fdaaea3ebc16d2eb0fdd38a38c8eaf04
1c8d45b081972d69b5dd79984f0f5b4d424ca7a6cad802c8a887350e3a92a745
NAMES      STATUS                    PORTS
interfaz   Up 12 seconds (healthy)   0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp
api        Up 37 seconds (healthy)   8080/tcp
{"estado":"listo","datos":"ok","modelo":"http://api:8080"}
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Tres líneas largas
Son los identificadores de lo que se acaba de crear: la red y los dos contenedores.

PORTS
La interfaz, `0.0.0.0:8090→8090/tcp`: publicada. La API, solo `8080/tcp`: sin puerta al equipo.

/listo
`"modelo":"http://api:8080"`: la interfaz llegó a la API **por su nombre**, dentro de la red.

ESCALÓN 4 · COMPROBACIÓN

Mismos números, otro sitio

Terminal del Codespace

```
$ curl -X POST http://127.0.0.1:8090/recomendar -H "Content-Type:
  application/json" -d '{"cliente_ids": [13599, 18108, 13306,
  99999]}'
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
origen http://api:8080 | corte 2011-12-10
13599 0.9977 alta puesto 1
18108 0.4159 media puesto 75
13306 0.128 baja puesto 148
99999 no existe en MASTER
```

Igual que en tu Windows

CLIENTE	ESCALÓN 1 (WINDOWS)	ESCALÓN 4 (CONTENEDORES)
13599	0,9977 · alta	0,9977 · alta
18108	0,4159 · media	0,4159 · media
13306	0,128 · baja	0,128 · baja

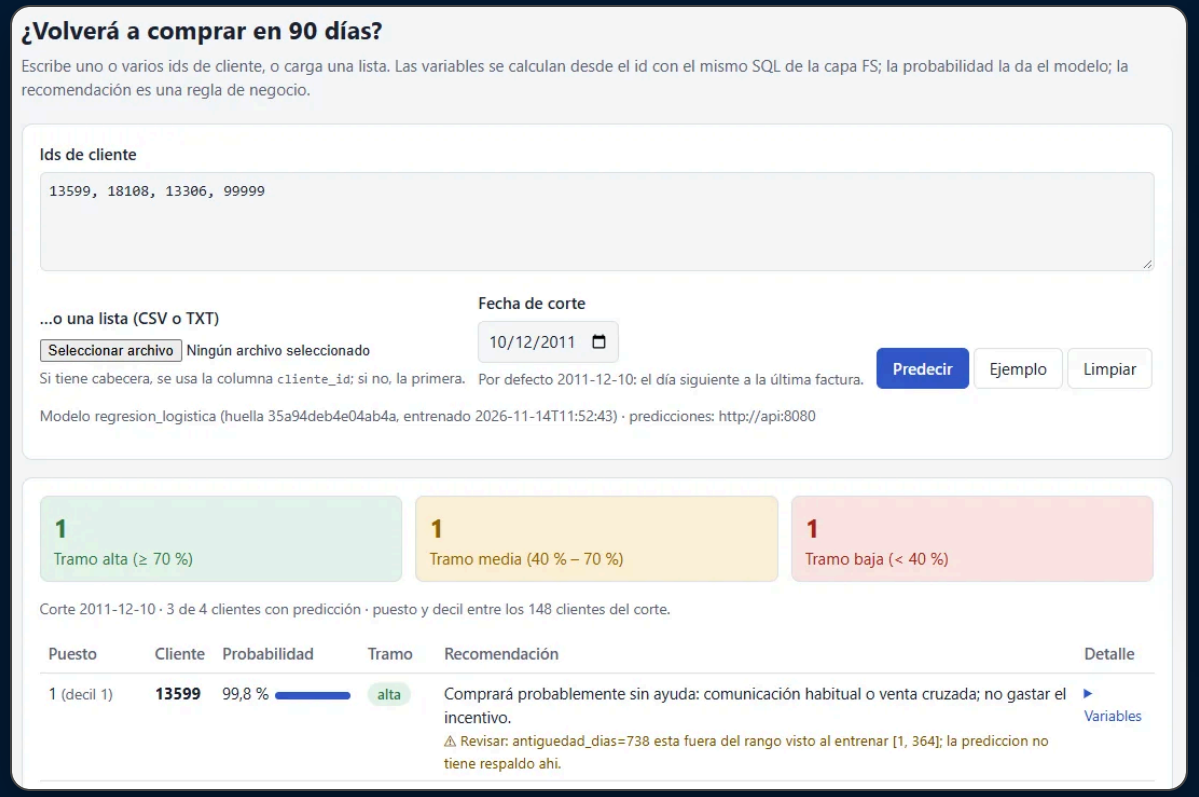
El modelo y los datos son los mismos: solo cambió **dónde corren**.

Abre la página: pestaña **Puertos** del Codespace → **8090**.

ASÍ SE VE EN TU CODESPACE · ESCALÓN 4

La misma respuesta, ahora desde los contenedores

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026



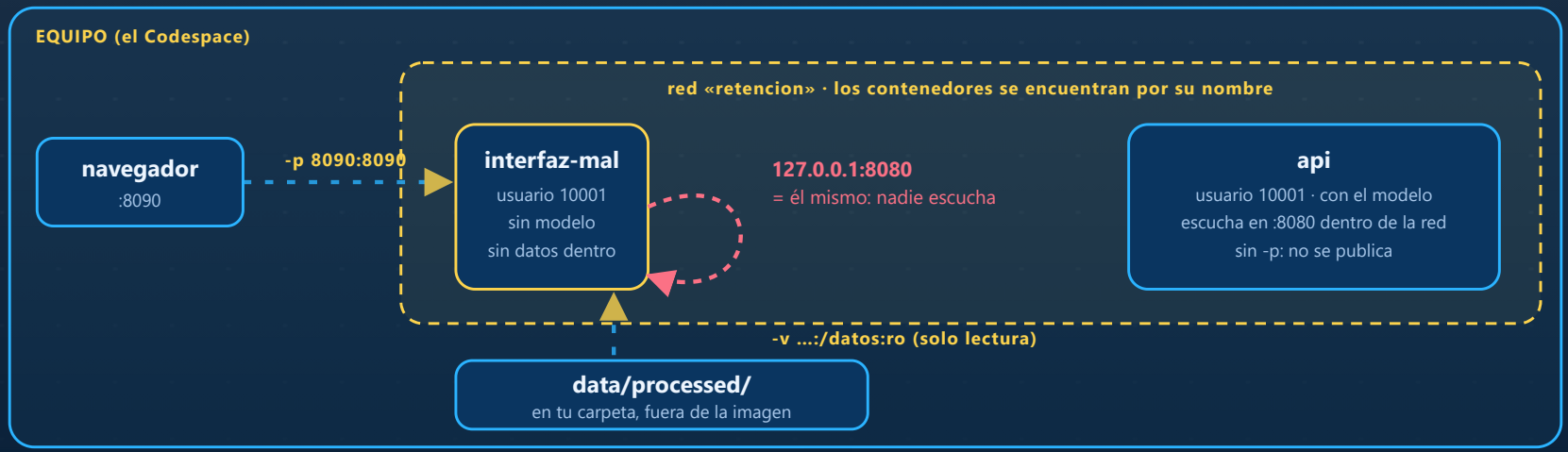

```
origen http://api:8080 | corte 2011-12-10
13599 0.9977 alta puesto 1
18108 0.4159 media puesto 75
13306 0.128 baja puesto 148
99999 no existe en MASTER
(env-ia-mlops) @helicr →/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Qué mirar

- Por `curl`: **13599** 0,9977 `alta` · **18108** 0,4159 `media` · **13306** 0,128 `baja` · **99999** no existe.
- Son **los mismos números** que en el escalón 1 y que en Windows.
- En la página (pestaña Puertos → 8090, recarga con F5): «predicciones: **http://api:8080**».
- Lo único que cambió es **dónde corren** las piezas.

ESCALÓN 4 · EL ERROR MÁS FRECUENTE

127.0.0.1 dentro de un contenedor es él mismo



Pruébalo

```
$ docker run -d --name interfaz-mal --network retencion -p 8091:8090 -e
  CURSO_URL_API=http://127.0.0.1:8080 -v
  "$PWD/data/processed:/datos:ro" retencion-interfaz:local
$ curl http://127.0.0.1:8091/listo
$ docker rm -f interfaz-mal
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
{"detalle":"La API del modelo en http://127.0.0.1:8080 no
responde ([Errno 111] Connection refused). ¿Está arrancada y
/listo responde?"}
```

ASÍ SE VE EN TU CODESPACE · ESCALÓN 4

El 503 de `127.0.0.1`, en tu pantalla

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
d70fa2d58d1f721b7eee998f11476d34d54ca36c525c94225a9dff8caa66ae40
{"detalle":"La API del modelo en http://127.0.0.1:8080 no responde ([Errno 111] Connection refused). ¿Está arrancada y /listo responde?"}
interfaz-mal
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Qué mirar

- La primera línea es el id del contenedor **`interfaz-mal`**.
- **`[Errno 111] Connection refused`**: dentro de **`interfaz-mal`** nadie escucha en el 8080.
- La última línea, **`interfaz-mal`**, es **`docker rm -f`** confirmando que lo borró.

Arreglo: dejar **`CURSO_URL_API`** en **`http://api:8080`** (el valor por defecto de la imagen) y que las dos piezas estén en la misma red.

ESCALÓN 4 · LABORATORIO

Romper a propósito: ¿qué esperas?

Escribir en los datos
esperado: falla

La API desde el equipo
esperado: no conecta

La API desde la interfaz
esperado: 200

Interfaz sin volumen
esperado: 503

Parar la API
esperado: 503

Parar la API

```
docker stop api
curl http://127.0.0.1:8090/salud     # 200
curl http://127.0.0.1:8090/listo
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
{"detalle":"La API del modelo en http://api:8080 no responde ([Errno -3]
Temporary failure in name resolution). ¿Está arrancada y /listo responde?"}
```

> Vivo no es listo. Y el motivo: al parar el contenedor, el nombre «api» desaparece de la red (en el Codespace el texto es «[Errno -2] Name or service not known»).

ASÍ SE VE EN TU CODESPACE · ESCALÓN 4

Las cinco roturas, una detrás de otra

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
--- 1. escribir en los datos
sh: 1: cannot create /datos/intruso: Read-only file system
--- 2. la API desde el equipo (sin -p)
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server

--- 3. la API desde la interfaz (por nombre)
b'{"estado":"listo"}'
--- 4. quien soy
10001
--- 5. parar la API
api
{"estado":"ok"}
{"detalle":"La API del modelo en http://api:8080 no responde ([Errno -2] Name or service not known). ¿Está arrancada y /listo responde?"}
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Qué mirar

- **1** `Read-only file system` · **2** `Could not connect to server` · **3** `{"estado":"listo"}`.
- **4** `10001`: el usuario sin privilegios · **5** `/salud` `{"estado":"ok"}` pero `/listo` 503.

Ojo: el motivo cambia según el sistema

Con la API parada, en el Codespace sale `[Errno -2] Name or service not known`; en GitHub Actions salió `[Errno -3] Temporary failure in name resolution`. Dicen lo mismo: **el nombre «api» ya no existe en la red**.

ESCALÓN 5 · UN ARCHIVO

Lo mismo, escrito una vez: Docker Compose

Terminal del Codespace · desde 04-Proyecto-Base

```
$ docker rm -f api interfaz; docker network rm retencion
$ docker compose -f deploy/docker/compose.yaml up --build -d
$ docker compose -f deploy/docker/compose.yaml ps
$ docker compose -f deploy/docker/compose.yaml logs -f interfaz
$ docker compose -f deploy/docker/compose.yaml down
```

SALIDA REAL · GITHUB ACTIONS · 14/11/2026

```
SERVICE    STATUS                    PORTS
api        Up 11 seconds (healthy)   8080/tcp
interfaz   Up 5 seconds (healthy)    0.0.0.0:8090→8090/tcp
```

La interfaz arrancó **6 s después**: Compose esperó a que la API estuviera sana (`depends_on: service_healthy`).

A MANO	EN COMPOSE.YAML
`docker network create retencion`	automática: `retencion_default` (proyecto + `_default`)
`--name api`	`services: api:`
`docker build -f …`	`build: {context, dockerfile}`
`-p 8090:8090`	`ports: ["8090:8090"]`
`-v …/data/processed:/datos:ro`	`volumes: ["../../data/processed:/datos:ro"]`
`-e CURSO_URL_API=…`	`environment:`
mirar `docker ps` hasta `(healthy)`	`depends_on: condition: service_healthy`

Compose sirve para **un equipo**. Para muchos, lo hacen plataformas como Cloud Run o Kubernetes (S3 y S4).

ASÍ SE VE EN TU CODESPACE · ESCALÓN 5

Compose: arriba, sano y abajo

CAPTURA REAL Codespace real del curso · rama develop · 14/11/2026

```
[+] Running 5/5
 ✓ retencion-api:local              Built                                  0.0s
 ✓ retencion-interfaz:local         Built                                  0.0s
 ✓ Network retencion_default        Created                                0.0s
 ✓ Container retencion-api-1        Healthy                                6.5s
 ✓ Container retencion-interfaz-1   Healthy                               11.4s
SERVICE    STATUS                     PORTS
api        Up 11 seconds (healthy)    8080/tcp
interfaz   Up 5 seconds (healthy)     0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp
{"estado":"listo","datos":"ok","modelo":"http://api:8080"}
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

```
api-1  | {"severity": "INFO", "message": "GET /salud 200", "evento": "peticion", "id": "c19fff093cec", "metodo": "GET", "ruta": "/salud", "codigo": 20
0, "ms": 0.9}
api-1  | INFO:     127.0.0.1:51854 - "GET /salud HTTP/1.1" 200 OK
api-1  | {"severity": "INFO", "message": "GET /salud 200", "evento": "peticion", "id": "eff9359d0845", "metodo": "GET", "ruta": "/salud", "codigo": 20
0, "ms": 0.7}
api-1  | INFO:     127.0.0.1:47632 - "GET /salud HTTP/1.1" 200 OK
interfaz-1  | INFO:     127.0.0.1:56514 - "GET /salud HTTP/1.1" 200 OK
interfaz-1  | INFO:     172.18.0.1:35296 - "GET /listo HTTP/1.1" 200 OK
interfaz-1  | INFO:     127.0.0.1:58532 - "GET /salud HTTP/1.1" 200 OK
interfaz-1  | INFO:     127.0.0.1:34952 - "GET /salud HTTP/1.1" 200 OK
[+] Running 3/3
 ✓ Container retencion-interfaz-1  Removed                                 0.4s
 ✓ Container retencion-api-1       Removed                                 0.7s
 ✓ Network retencion_default       Removed                                 0.0s
(env-ia-mlops) @helicr ➜/workspaces/curso-ml-ia-desarrollo-produccion/04-Proyecto-Base (develop) $
```

Qué mirar

- `up --build -d --wait`: **Built**, red **`retencion_default`** creada y los dos contenedores **Healthy**.
- La red la nombra Compose: proyecto `retencion` + `_default`.
- `ps`: los mismos puertos que a mano; `/listo` con `http://api:8080`.
- `logs`: cada línea lleva delante su servicio (`api-1`, `interfaz-1`).
- `down`: quita contenedores y red. **Las imágenes y los datos se quedan.**

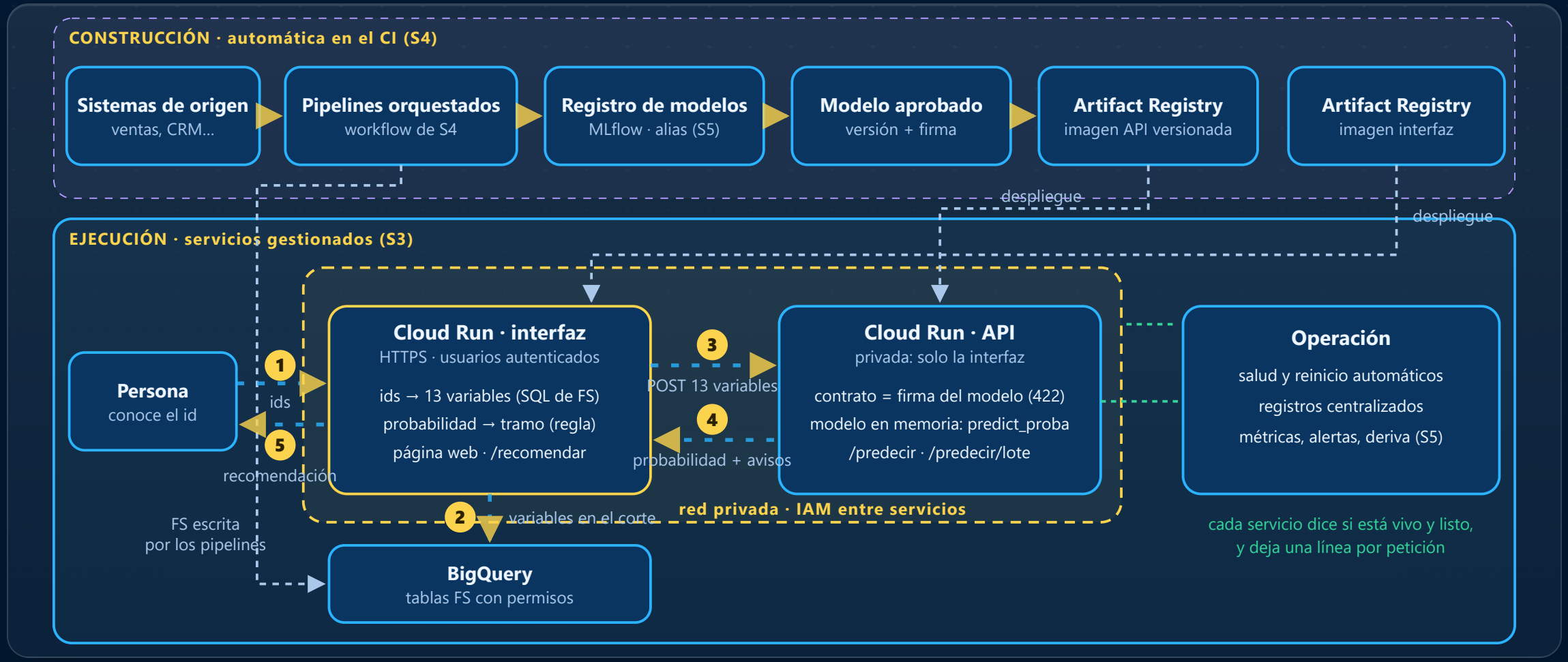
ESCALÓN 6 · HONESTIDAD

Ya tienes el artefacto. El Codespace no es producción

ASPECTO	EN EL CODESPACE (HOY)	EN PRODUCCIÓN	EN EL CURSO
Dónde corre	una máquina tuya que se apaga	plataforma gestionada que lo mantiene y reinicia	S3 · Cloud Run
Quién llega	solo tú (puerto privado)	usuarios autenticados; la API solo desde servicios autorizados	S3 · IAM
Imágenes	construidas a mano	construidas en el CI, versionadas en un registro	S3–S4 · Artifact Registry
Datos	SQLite montado como volumen	almacén gestionado, con permisos y copias	S3 · BigQuery
Secretos	no hay	en un gestor de secretos, nunca en la imagen	S3 · Secret Manager
Despliegue	`docker compose up`	automático, con pruebas y aprobación	S4 · CI/CD
Vigilancia	`docker logs`	registros, métricas y alertas	S3 y S5
Calidad del modelo	el aviso «revisar»	deriva, reentrenamiento y vuelta atrás	S5

Pasar a producción **no cambia el código ni las imágenes**: cambia **dónde** corren y **lo que las rodea**. Por eso hoy separamos las piezas, sacamos los datos y pusimos la configuración fuera.

La misma arquitectura, en producción



RESUMEN

Los escalones, de un vistazo

	QUÉ CORRE	DEPENDE DE	QUIÉN LLEGA	DATOS
1 · Mi ordenador	dos procesos Python	tu conda, tus archivos, tu sesión	solo tú (`127.0.0.1`)	tu carpeta, con escritura
2 · API en imagen	un contenedor	solo la imagen	tú, por `-p 8080`	no usa
3–4 · Dos contenedores	dos contenedores en una red	solo las imágenes	tú, solo a la interfaz	volumen `:ro`
5 · Compose	lo mismo, en un archivo	el archivo y las imágenes	igual	igual
6 · Producción	las mismas imágenes	la plataforma	usuarios autenticados	almacén gestionado

Del 1 al 2: **reproducible**. Del 2 al 4: **separado y protegido**.

Del 4 al 5: **documentado**. Del 5 al 6: **gestionado y vigilado**.

COMPRUEBA QUE LO TIENES

Seis preguntas

1. ¿Por qué la interfaz llama a `http://api:8080` y no a `127.0.0.1`?
Dentro de un contenedor, `127.0.0.1` es él mismo. En la red, Docker resuelve `api` al contenedor de la API.

2. ¿Por qué la API no lleva `-p`?
Nadie de fuera necesita hablarle: solo la interfaz, dentro de la red. Exponer lo mínimo.

3. ¿Por qué los datos van en un volumen?
La imagen se copia y se comparte: no debe llevar datos de clientes. Con `:ro`, no se pueden estropear.

4. La API se cae: ¿qué dan `/salud` y `/listo`?
`/salud` 200 y `/listo` 503. Si el `HEALTHCHECK` mirara `/listo`, Docker reiniciaría la interfaz sin arreglar nada.

5. ¿Qué cambia en el código del escalón 1 al 4? ¿Y al 6?
Nada. Cambian la configuración y dónde corren las piezas.

6. ¿Cuándo NO merece la pena llevar un modelo a producción?
Cuando es un análisis puntual o un prototipo que nadie más usará ni repetirá.

CIERRE

Qué te llevas

Hoy

- Por qué (y cuándo) sacar un modelo del **portátil**
- Una **interfaz** que convierte ids en recomendaciones
- **Dos imágenes**: la del modelo y la de la interfaz, sin datos
- **Red, puertos, volumen `:ro`** y vivo frente a listo
- **Compose** y lo que falta para producción

Para seguir

- Guía completa: `docs/del-ordenador-al-servicio.md`
- Recorrido de clase: `practica/interfaz-de-consulta.md`
- Al acabar: `docker compose -f deploy/docker/compose.yaml down`
- S3: esta imagen, en **Cloud Run**

Probado con Docker real en GitHub Actions y recorrido entero en un Codespace del curso (14/11/2026). Si algo te sale distinto, anótalo.